Composable semantic views

What are composable semantic views?

Composable semantic views introduce an `IMPORTS` clause that lets one semantic view pull in the full set of tables, dimensions, facts, metrics, and relationships from another semantic view. This creates a reusable, shared semantic layer where common business definitions are authored once and referenced by many downstream views.

Without composability, teams that need the same customer, account, or calendar definitions must duplicate them in every semantic view. When definitions drift apart, queries return inconsistent results and maintenance costs increase. Composable semantic views solve this by establishing a single source of truth.

Use cases

- **Shared dimensions**: Define customer, account, product, geography, or calendar entities in one base semantic view. Domain teams (sales, marketing, finance) import the base view and add their own local facts and metrics.
- **Centralized governance**: Business-critical definitions (revenue, churn, active users) are owned by a central analytics team. Downstream consumers import these definitions and can't accidentally alter them.
- **Modular semantic layers**: Large organizations can decompose a monolithic semantic view into focused modules (one per domain) that compose together, reducing complexity and improving change isolation.
- **Cross-domain queries**: A composed view that imports both a customer dimension and an orders domain lets users query revenue by customer segment without writing joins or understanding the physical schema.

SQL syntax

CREATE SEMANTIC VIEW with IMPORTS

```
CREATE [ OR REPLACE ] SEMANTIC VIEW <name>
  IMPORTS ( <semantic_view_name> [ , ... ] )
  [ TABLES ( ... ) ]
  [ RELATIONSHIPS ( ... ) ]
  [ FACTS ( ... ) ]
  [ DIMENSIONS ( ... ) ]
  [ METRICS ( ... ) ]
  ...
```

The `IMPORTS` clause appears before the `TABLES` clause. You can import one or more semantic views. A composed view can also omit `TABLES` entirely if it only combines imported entities.

Key rules:

- Imports are all-or-nothing: when you import a semantic view, all of its entities are brought in. You can't selectively import individual tables or calculations, and name collisions between imports aren't merged automatically.
- A composed view can only reference imported calculations (dimensions, facts, metrics). It can't reference the physical columns of an imported entity directly.
- Imported entities are referenced by their logical table name and calculation name (for example, `customers.d_region` or `orders.m_total`), the same way you reference local objects.
- Relationships between imported entities use logical dimension or fact names, not physical column names.
- A composed view can define local tables alongside imports.
- A composed view can define new metrics on imported entities.

Transitive imports

Import chains are resolved transitively. If view A imports view B, and view B imports view C, then view A has access to all entities from both B and C:

```
CREATE OR REPLACE SEMANTIC VIEW sv_mid IMPORTS (sv_orders);
CREATE OR REPLACE SEMANTIC VIEW sv_top IMPORTS (sv_mid);

-- Queries entities originally defined in sv_orders:
SELECT * FROM SEMANTIC_VIEW(sv_top METRICS orders.m_total);
```

Runtime behavior

A composed view resolves imported objects at query time (late binding). It doesn't freeze a snapshot of the source view's definitions. If the source is altered to remove an object that the composed view references, the query fails with error `000904` (invalid identifier).

To fix this, restore the missing object in the source view or update the query.

Privilege resolution

To create a composed view, the role must have both REFERENCES and SELECT privileges on each directly imported semantic view.

At query time, imported base tables resolve under the source semantic view's owner role, not the caller's role. A user can query a composed view without direct access to the underlying base tables, as long as they have the appropriate privileges on the composed view itself.

YAML format

Composable semantic views are supported in YAML. Use the `imports` top-level block to specify which semantic views to import:

```
imports:
  - database_name: <DB>
    schema_name: <SCHEMA>
    name: <SEMANTIC_VIEW_NAME>
```

When a composed view defines a local calc on an imported entity, the exported table entry includes `is_imported: true` to mark it as a shadow of the imported entity:

```yaml
tables:
  - name: ORDERS
    is_imported: true
    metrics:
      - name: M_DOUBLE_TOTAL
        expr: orders.m_total * 2
        access_modifier: public_access
```

The export also includes an `imported_semantic_models` block containing the full YAML body of every semantic view in the transitive import closure.

Metadata and introspection

DESCRIBE SEMANTIC VIEW

`DESCRIBE SEMANTIC VIEW` exposes composition through additional row types:

- **IMPORT rows**: One row per directly imported semantic view, with columns `IMPORTED_SEMANTIC_VIEW_DATABASE_NAME`, `IMPORTED_SEMANTIC_VIEW_SCHEMA_NAME`, and `IMPORTED_SEMANTIC_VIEW_NAME`.
- **Shadow TABLE rows**: Appear when the composed view defines a local calc on an imported entity. These rows have empty `BASE_TABLE_*` values.
- **Calculations on shadows**: Normal METRIC/DIMENSION rows attached to shadow tables.

```
DESCRIBE SEMANTIC VIEW sv_composed;
```

SHOW SEMANTIC VIEWS

The `SHOW SEMANTIC VIEWS` command includes an `imports` column that lists direct imports as an array of fully qualified names. Non-composed views show an empty array (`[]`).

```
SHOW SEMANTIC VIEWS LIKE 'SV_COMPOSED';
-- imports column: ["MY_DB.PUBLIC.SV_CUSTOMERS","MY_DB.PUBLIC.SV_ORDERS"]

SHOW SEMANTIC VIEWS LIKE 'SV_CUSTOMERS';
-- imports column: []
```

GET_DDL

`GET_DDL` output includes the `IMPORTS` clause with fully qualified semantic view names:

```
SELECT GET_DDL('SEMANTIC_VIEW', 'sv_composed');
```

SYSTEM$READ_YAML_FROM_SEMANTIC_VIEW

See YAML format above for the full output structure.

```
SELECT SYSTEM$READ_YAML_FROM_SEMANTIC_VIEW('sv_composed');
```

Limitations

- **Cortex Analyst**: %cortex-analyst% doesn't support semantic views that use the `IMPORTS` clause. Queries sent through Cortex Analyst that reference a composed semantic view will fail.

- **Snowsight UI**: The Snowsight UI doesn't support semantic views with imports. Attempting to view or manage a composed semantic view in the UI generates an error.
- **Replication**: Semantic views that use the `IMPORTS` clause aren't replicated. They are skipped during account or database replication.
- Imports are all-or-nothing. You can't selectively import individual tables or calculations from a source view. Name collisions between imported views aren't merged; they must be resolved before creation.
- A composed view can only reference imported calculations (dimensions, facts, metrics), not the physical columns of imported entities.
- Diamond import graphs (the same source view reachable through multiple paths) aren't supported.
- Relationships on imported entities must reference logical names (dimensions or facts), not physical column names from the imported base table.

Example: shared account dimension across domains

This end-to-end example creates a shared account dimension and imports it into two domain-specific semantic views (sales and marketing), then runs queries that combine imported and local objects.

```
-- === Physical tables ===

CREATE OR REPLACE TABLE accounts (
  account_id INT,
  account_name VARCHAR,
  industry VARCHAR,
  region VARCHAR
);
CREATE OR REPLACE TABLE sales (
  sale_id INT,
  account_id INT,
  amount NUMBER,
  sale_date DATE
);
CREATE OR REPLACE TABLE campaigns (
  campaign_id INT,
  account_id INT,
  spend NUMBER,
  impressions INT
);

INSERT INTO accounts VALUES
  (1, 'Acme Corp', 'Technology', 'US'),
  (2, 'Globex Inc', 'Finance', 'EU'),
  (3, 'Initech', 'Technology', 'US');
INSERT INTO sales VALUES
  (101, 1, 50000, '2025-01-15'),
  (102, 2, 75000, '2025-02-20'),
```

```
  (103, 1, 30000, '2025-03-10'),
  (104, 3, 45000, '2025-03-22');
INSERT INTO campaigns VALUES
  (201, 1, 5000, 10000),
  (202, 2, 8000, 25000),
  (203, 3, 3000, 7000);

-- === Base semantic view: shared account dimension ===

CREATE OR REPLACE SEMANTIC VIEW sv_account_dimension
TABLES (
  accounts AS accounts PRIMARY KEY (d_account_id)
)
DIMENSIONS (
  accounts.d_account_id AS account_id,
  accounts.d_account_name AS account_name
    WITH SYNONYMS = ('company', 'client'),
  accounts.d_industry AS industry,
  accounts.d_region AS region
)
METRICS (
  accounts.m_account_count AS COUNT(account_id)
);

-- === Sales domain: imports the shared account dimension ===

CREATE OR REPLACE SEMANTIC VIEW sv_sales
IMPORTS (sv_account_dimension)
TABLES (
  sales AS sales PRIMARY KEY (sale_id)
)
DIMENSIONS (
  sales.d_sale_date AS sale_date,
  sales.d_account_id AS account_id
)
METRICS (
  sales.m_total_revenue AS SUM(amount),
  sales.m_deal_count AS COUNT(sale_id)
)
RELATIONSHIPS (
  sales_to_accounts AS sales(d_account_id) REFERENCES accounts
);

-- === Marketing domain: imports the same shared account dimension ===

CREATE OR REPLACE SEMANTIC VIEW sv_marketing
IMPORTS (sv_account_dimension)
TABLES (
  campaigns AS campaigns PRIMARY KEY (campaign_id)
```

```
)
DIMENSIONS (
  campaigns.d_campaign_id AS campaign_id,
  campaigns.d_account_id AS account_id
)
METRICS (
  campaigns.m_total_spend AS SUM(spend),
  campaigns.m_total_impressions AS SUM(impressions)
)
RELATIONSHIPS (
  campaigns_to_accounts AS campaigns(d_account_id) REFERENCES accounts
);

-- === Queries ===

-- Revenue by industry (imported dimension + local metric)
SELECT * FROM SEMANTIC_VIEW(
  sv_sales
  DIMENSIONS accounts.d_industry
  METRICS sales.m_total_revenue
);
-- Expected: Technology = 125000, Finance = 75000

-- Spend by region (imported dimension + local metric)
SELECT * FROM SEMANTIC_VIEW(
  sv_marketing
  DIMENSIONS accounts.d_region
  METRICS campaigns.m_total_spend
);
-- Expected: US = 8000, EU = 8000

-- Account count from imported metric
SELECT * FROM SEMANTIC_VIEW(
  sv_sales
  METRICS accounts.m_account_count
);
-- Expected: 3

-- Mix imported dimension with local dimension
SELECT * FROM SEMANTIC_VIEW(
  sv_sales
  DIMENSIONS accounts.d_account_name, sales.d_sale_date
  METRICS sales.m_total_revenue
);
```